Name of Student: Vyankatesh Rohokale	**PRN: 26070122518**
Semester: III	**Year: AY 2026-27**
Subject: Programming with Java	
ASSIGNMENT No: 27	
TITLE: Develop a Java application demonstrating socket programming for communication between a client and a server.	

Problem Statement

Develop a Java application demonstrating socket programming for communication between a client and a server.

Learning Objectives

To establish communication between client and server using Java sockets.

Programme

1.Create a simple client-server chat application using sockets.

1: ChatServer.java:

```
import java.net.*;

import java.io.*;


public class ChatServer {

    public static void main(String[] args) throws Exception {

        ServerSocket server = new ServerSocket(5000);

        System.out.println("Server waiting for client...");


        Socket socket = server.accept();

        System.out.println("Client connected");


        BufferedReader input = new BufferedReader(
```

```
            new InputStreamReader(socket.getInputStream())
        );

        PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader keyboard = new BufferedReader(
            new InputStreamReader(System.in)
        );

        String message;

        while (true) {
            message = input.readLine();

            if (message.equalsIgnoreCase("exit")) {
                break;
            }

            System.out.println("Client: " + message);
            System.out.print("Server reply: ");
            output.println(keyboard.readLine());
        }

        socket.close();
        server.close();
    }
}
```

2 : ChatClient.java

```
import java.net.*;
import java.io.*;

public class ChatClient {
    public static void main(String[] args) throws Exception {
        Socket socket = new Socket("localhost", 5000);

        BufferedReader input = new BufferedReader(
            new InputStreamReader(socket.getInputStream())
        );

        PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader keyboard = new BufferedReader(
            new InputStreamReader(System.in)
        );

        String message;

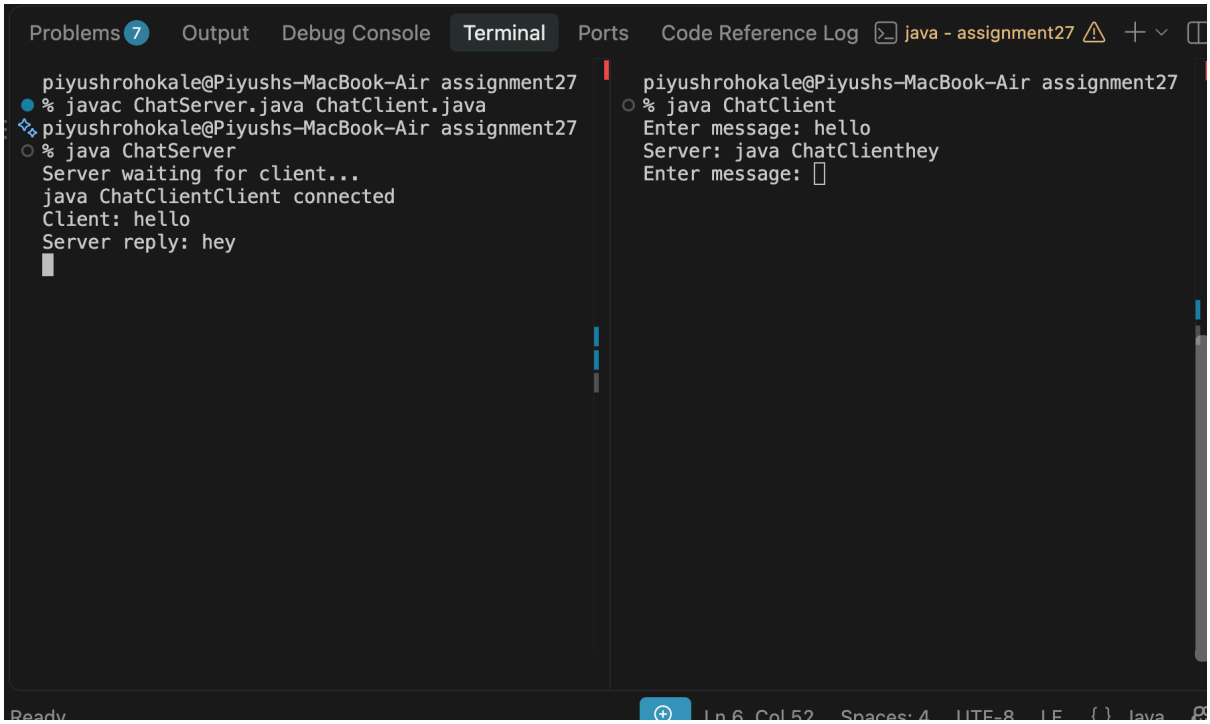
        while (true) {
            System.out.print("Enter message: ");
            message = keyboard.readLine();
            output.println(message);

            if (message.equalsIgnoreCase("exit")) {
                break;
```

```
        }

        System.out.println("Server: " + input.readLine());
    }

    socket.close();
  }
}
```



2. Create a Java program using socket programming where the client sends a message to the server, and the server returns the same message back (echo system).

1:EchoServer.java:

```
import java.net.*;
import java.io.*;

public class EchoServer {
    public static void main(String[] args) throws Exception {
        ServerSocket server = new ServerSocket(5001);
        System.out.println("Echo server waiting for client...");

        Socket socket = server.accept();
        System.out.println("Client connected");

        BufferedReader input = new BufferedReader(
            new InputStreamReader(socket.getInputStream())
        );

        PrintWriter output = new PrintWriter(socket.getOutputStream(), true);

        String message;

        while ((message = input.readLine()) != null) {
            if (message.equalsIgnoreCase("exit")) {
```

```
            break;
        }

        System.out.println("Received: " + message);
        output.println(message);
    }

    socket.close();
    server.close();
  }
}
```

2: EchoClient.java:

```
import java.net.*;
import java.io.*;

public class EchoClient {
  public static void main(String[] args) throws Exception {
    Socket socket = new Socket("localhost", 5001);

    BufferedReader keyboard = new BufferedReader(
```

```
            new InputStreamReader(System.in)
        );

        BufferedReader input = new BufferedReader(
            new InputStreamReader(socket.getInputStream())
        );

        PrintWriter output = new PrintWriter(socket.getOutputStream(), true);

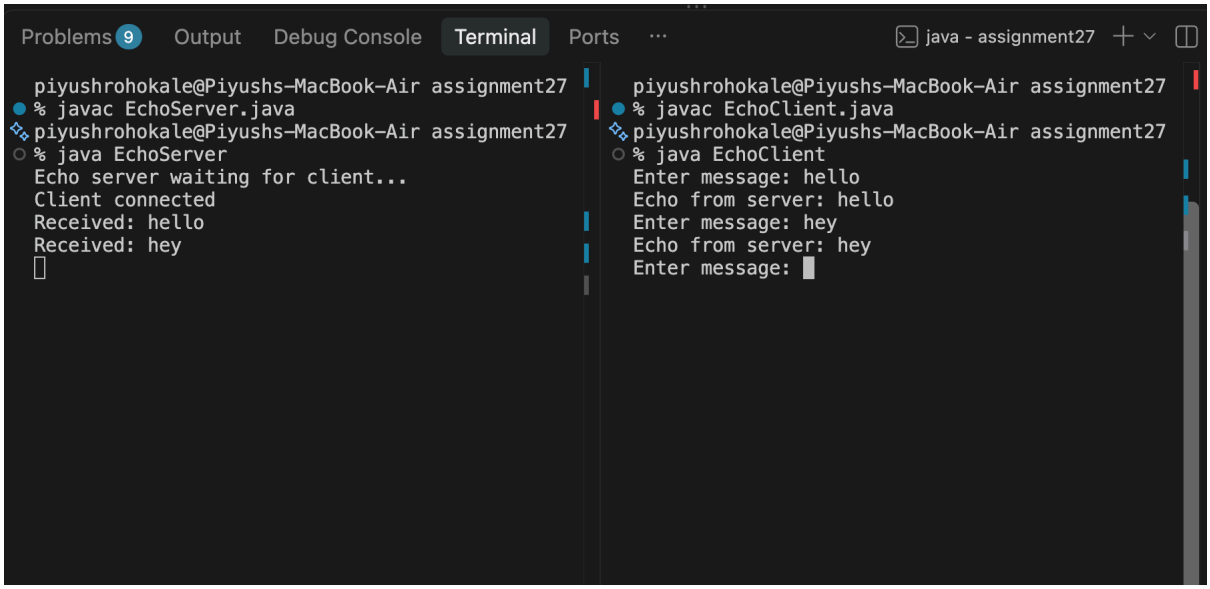
        String message;

        while (true) {
            System.out.print("Enter message: ");
            message = keyboard.readLine();
            output.println(message);

            if (message.equalsIgnoreCase("exit")) {
                break;
            }

            System.out.println("Echo from server: " + input.readLine());
        }

        socket.close();
    }
}
```



Github :



Conclusion:

The client sends a message to the server, and the server returns the same message to the client through the socket connection.